

Top 50 .NET Interview Questions & Answers

For Freshers & Experienced Professionals

Covers .NET Framework vs .NET Core, CLR, C# fundamentals, OOP concepts, Entity Framework & Migrations, ASP.NET Core MVC/Web API, LINQ, dependency injection, async programming, and more.

Section 1: Fresher-Level Questions (Fundamentals)

Q1. What is .NET?

.NET is a free, open-source developer platform created by Microsoft for building many kinds of applications — web, mobile, desktop, gaming, IoT, and cloud services. It includes a runtime (CLR), a set of class libraries (BCL/FCL), and languages such as C#, F#, and VB.NET.

Q2. What is the difference between .NET Framework and .NET Core (.NET 5+)?

.NET Framework is the original, Windows-only platform (legacy, in maintenance mode). .NET Core (now unified simply as ".NET" from .NET 5 onward) is cross-platform, open-source, faster, modular, and supports side-by-side versioning. New application development is generally done on modern .NET rather than .NET Framework, since Framework only receives limited support going forward.

Q3. What is CLR?

The Common Language Runtime (CLR) is the execution engine of .NET. It handles memory management (garbage collection), type safety, exception handling, thread management, and Just-In-Time (JIT) compilation of Intermediate Language (IL) code into native machine code.

Q4. What is IL (Intermediate Language)?

IL, also called MSIL or CIL, is the CPU-independent set of instructions that .NET compilers (like the C# compiler) produce. The CLR's JIT compiler converts IL into native code at runtime, which is what allows any .NET language to run on any platform supported by the CLR.

Q5. What is the Base Class Library (BCL)?

The BCL is the core set of built-in classes, interfaces, and value types shipped with .NET that provide fundamental functionality such as collections, file I/O, string handling, networking, and threading, so developers don't have to write these from scratch.

Q6. What is the difference between value types and reference types?

Value types (e.g., int, struct, bool, enum) store their data directly and are usually allocated on the stack; copying a value type copies the data. Reference types (e.g., class, string, array, delegate) store a reference to data on the heap; copying a reference type copies the reference, so both variables point to the same object.

Q7. What is Object-Oriented Programming (OOP), and what are its main pillars?

OOP is a programming paradigm based on organizing code into objects. Its four pillars are: Encapsulation (bundling data and methods, hiding internal state), Abstraction (exposing only essential details), Inheritance (reusing behavior from a base class), and Polymorphism (treating objects of different types through a common interface, with behavior resolved at compile or run time).

Q8. What is the difference between an abstract class and an interface?

An abstract class can provide default implementation, fields, and constructors, and a class can inherit from only one abstract class. An interface (traditionally) only declares members with no implementation (though C# 8+ allows default interface methods), and a class can implement multiple interfaces. Use an abstract class for a shared base with common code; use an interface for a contract across unrelated types.

Q9. What is the difference between method overloading and method overriding?

Overloading means defining multiple methods with the same name but different parameter lists within the same class — resolved at compile time. Overriding means a derived class redefines a

virtual/abstract method from its base class using the same signature — resolved at run time (polymorphism).

Q10. What are the differences between 'ref', 'out', and 'in' parameters in C#?

'ref' passes a variable by reference and requires it to be initialized before the call. 'out' also passes by reference but does not require prior initialization — the callee must assign it before returning. 'in' passes by reference as read-only, preventing the callee from modifying the argument.

Q11. What is boxing and unboxing?

Boxing is converting a value type into a reference type (object), which involves allocating memory on the heap. Unboxing is extracting the value type back out of that object. Both operations have a performance cost, so they should be avoided in performance-critical code where possible.

Q12. What is a namespace in C#?

A namespace is a way to organize and group related classes, interfaces, and other types, and to avoid naming conflicts between types that share the same name in different parts of a codebase or libraries.

Q13. What is garbage collection in .NET?

Garbage collection (GC) is the automatic memory management process performed by the CLR. It periodically identifies objects on the managed heap that are no longer reachable by the application and reclaims their memory, so developers generally do not need to manually free memory.

Q14. What is the difference between 'const' and 'readonly'?

'const' is a compile-time constant; its value must be assigned at declaration and never changes. 'readonly' can be assigned either at declaration or inside a constructor, allowing different instances to have different fixed values, and its value is resolved at run time.

Q15. What is exception handling, and how is it done in C#?

Exception handling is a mechanism to gracefully deal with runtime errors using try, catch, and finally blocks. Code that might throw an error goes in the try block, catch blocks handle specific exception types, and the finally block runs cleanup code regardless of whether an exception occurred.

Q16. What is the difference between 'String' and 'StringBuilder'?

'String' is immutable — every modification creates a new string object in memory. 'StringBuilder' is mutable and is more efficient when performing many string concatenations or modifications in a loop, since it avoids creating multiple intermediate string objects.

Q17. What are collections in .NET, and can you name a few?

Collections are types used to store and manage groups of related objects. Common examples include List<T>, Dictionary<TKey, TValue>, Queue<T>, Stack<T>, and HashSet<T>, all found in the System.Collections.Generic namespace.

Q18. What is LINQ?

LINQ (Language Integrated Query) is a set of features that lets developers write strongly-typed, SQL-like queries directly in C# against collections, databases (via Entity Framework), XML, and other data sources, using methods like Where, Select, OrderBy, and GroupBy.

Q19. What is the difference between IEnumerable and IQueryable?

IEnumerable executes queries in-memory and is best for working with local collections; it loads data first and then filters. IQueryable builds an expression tree that can be translated (e.g., to SQL) and

executed on the data source itself, making it more efficient for querying databases since filtering happens before data is retrieved.

Q20. What is an assembly in .NET?

An assembly is the basic unit of deployment in .NET — a compiled DLL or EXE file containing IL code, metadata, and a manifest describing the assembly's identity and dependencies.

Section 2: Experienced-Level Questions

Q21. What is Entity Framework (EF) / EF Core?

Entity Framework is Microsoft's Object-Relational Mapper (ORM) for .NET. It lets developers work with a database using C# classes and LINQ instead of writing raw SQL, automatically translating queries into SQL and mapping results back into objects. EF Core is the modern, cross-platform, lightweight rewrite of Entity Framework used with .NET Core/.NET 5+.

Q22. What are migrations in Entity Framework?

Migrations are a way to incrementally evolve the database schema to keep it in sync with the application's data model (entity classes) over time, without dropping or manually recreating the database. Each migration is a code file recording the schema changes (add/remove tables or columns) needed to move from one model version to the next; commands like 'Add-Migration' generate them and 'Update-Database' (or 'dotnet ef database update') applies them.

Q23. What is the difference between Code-First, Database-First, and Model-First approaches in EF?

Code-First starts from C# entity classes, and EF generates the database schema and migrations from them — the most common modern approach. Database-First starts from an existing database, and EF scaffolds entity classes and a context from it. Model-First uses a visual designer to create a model that generates both the database and the classes; it is largely legacy and rarely used today.

Q24. What is the difference between 'Add-Migration' and 'Update-Database'?

'Add-Migration' generates a new migration file capturing the differences between the current model and the last recorded snapshot. 'Update-Database' applies pending migrations to the actual database, executing the SQL needed to bring its schema up to date.

Q25. What is lazy loading, eager loading, and explicit loading in EF?

Lazy loading defers fetching related data until it is actually accessed, potentially causing multiple extra queries. Eager loading loads related data upfront in the same query using 'Include()', reducing round-trips. Explicit loading loads related data on demand, but only when the developer calls a load method explicitly (e.g., 'context.Entry(entity).Collection(...).Load()').

Q26. What is the difference between .NET Standard and .NET Core?

.NET Standard is a specification — a set of APIs that multiple .NET implementations (Framework, Core, Xamarin, Mono) agree to support, used to write shared libraries. .NET Core (now just '.NET') is an actual runtime and SDK implementation. From .NET 5 onward, .NET Standard's role has largely been replaced by targeting .NET directly, since all implementations converged into one platform.

Q27. What is Dependency Injection (DI), and how does ASP.NET Core support it?

Dependency Injection is a design pattern where an object's dependencies are provided externally rather than created internally, improving testability and loose coupling. ASP.NET Core has a built-in DI container: services are registered in 'Program.cs' (or 'Startup.cs') with a lifetime — Transient (new instance every time), Scoped (one instance per request), or Singleton (one instance for the app's lifetime) — and then injected into constructors automatically.

Q28. What is Middleware in ASP.NET Core?

Middleware are components assembled into a request pipeline that handle HTTP requests and responses. Each middleware can process a request, pass it to the next component via 'next()', and process the response on the way back. Examples include authentication, routing, exception handling,

and logging middleware, configured in 'Program.cs' via 'app.Use...' calls.

Q29. What is the difference between ASP.NET MVC and ASP.NET Core MVC?

ASP.NET MVC (classic) runs only on .NET Framework and Windows/IIS. ASP.NET Core MVC is a rewrite that runs on modern .NET, is cross-platform, has built-in dependency injection, unifies MVC and Web API into one framework, and is generally faster with a more modular, configurable pipeline.

Q30. What is the difference between Web API and WCF?

Web API is a lightweight framework for building HTTP-based RESTful services that return data in JSON or XML, ideal for web and mobile clients. WCF (Windows Communication Foundation) is a more heavyweight, configuration-heavy framework supporting multiple protocols (HTTP, TCP, MSMQ) and messaging patterns, commonly used for enterprise SOAP-based services, though it is largely legacy for new .NET development.

Q31. What is async/await, and why is it used?

'async' and 'await' allow writing asynchronous code that reads like synchronous code. Marking a method 'async' lets it use 'await' on a Task, which frees up the calling thread while the awaited operation (like an I/O call) completes, improving scalability and responsiveness — especially important in web servers handling many concurrent requests without blocking threads.

Q32. What is the difference between Task and Thread in C#?

A Thread represents an actual OS-level thread of execution and is a lower-level, more expensive construct to manage directly. A Task represents an asynchronous operation and is a higher-level abstraction that uses the thread pool efficiently, supports composition (continuations, WhenAll), and better integrates with async/await.

Q33. What is the difference between .NET's Singleton, Scoped, and Transient service lifetimes?

Transient services are created each time they are requested — best for lightweight, stateless services. Scoped services are created once per client request (or scope) and reused within that scope — common for things like a database context. Singleton services are created once for the entire application lifetime and shared across all requests, so they must be thread-safe.

Q34. What is the repository pattern, and why is it used?

The repository pattern abstracts data access logic behind an interface, so the rest of the application interacts with a repository rather than directly with EF or a database. This improves testability (by allowing mocked repositories), separates concerns, and centralizes query logic, though with modern EF Core some consider it partly redundant since DbContext already implements similar patterns.

Q35. What is the difference between optimistic and pessimistic concurrency in EF?

Optimistic concurrency assumes conflicts are rare: it checks (often via a row-version/timestamp column) whether data changed since it was read, and throws a 'DbUpdateConcurrencyException' if a conflict is detected at save time. Pessimistic concurrency locks the data as soon as it's read, preventing others from modifying it until the lock is released, which reduces conflicts but can hurt scalability.

Q36. What is a middleware pipeline order issue, and why does ordering matter in ASP.NET Core?

Middleware executes in the exact order it is registered, and each middleware can short-circuit the pipeline. For example, if authentication middleware is registered after authorization middleware,

requests may be evaluated before the user's identity is established, causing incorrect behavior. Correct ordering (e.g., exception handling first, then routing, authentication, authorization, then endpoints) is essential for the pipeline to work as intended.

Q37. What is the difference between *AddScoped*, *AddTransient*, and *AddSingleton* when registering *DbContext*?

DbContext is typically registered as *Scoped* (via '*AddDbContext*') so that a single instance is shared throughout one HTTP request, keeping the same change-tracking context for related operations. Using *Singleton* would keep the same *DbContext* across the whole app's lifetime, which is unsafe since *DbContext* is not thread-safe. *Transient* would create a new instance every time it's injected, potentially breaking unit-of-work behavior within a single request.

Q38. What are *Data Annotations* and *Fluent API* in EF, and how do they differ?

Both configure how entity classes map to database schema. *Data Annotations* are attributes placed directly on entity properties (e.g., '*[Required]*', '*[MaxLength(50)]*') — simple and readable for basic rules. *Fluent API* is configured in code inside '*OnModelCreating*', offering more power and flexibility for complex configurations (like composite keys or relationships) that annotations cannot express.

Q39. What is the difference between *synchronous* and *asynchronous EF Core methods* (e.g., *SaveChanges* vs *SaveChangesAsync*)?

'*SaveChanges*' blocks the calling thread until the database operation completes. '*SaveChangesAsync*' performs the same operation but frees the thread to handle other work while waiting on I/O, which improves throughput and scalability in web applications handling many concurrent requests.

Q40. What is *Kestrel* in ASP.NET Core?

Kestrel is the default, cross-platform web server built into ASP.NET Core. It's lightweight and fast, and is often used behind a reverse proxy such as IIS, Nginx, or Apache in production for added security and features like load balancing.

Q41. What is the difference between *appsettings.json* and *environment variables* for configuration?

'*appsettings.json*' stores structured configuration values as part of the project and supports environment-specific overrides (e.g., '*appsettings.Development.json*'). Environment variables are set at the OS or hosting-platform level and are commonly used to override settings per deployment environment or to keep sensitive values (like connection strings) out of source control.

Q42. What is *CORS*, and how is it handled in ASP.NET Core?

CORS (Cross-Origin Resource Sharing) is a browser security feature that restricts web pages from making requests to a different domain than the one that served the page. In ASP.NET Core, CORS policies are configured in '*Program.cs*' by defining allowed origins, headers, and methods, and then applying the policy globally or to specific endpoints/controllers.

Q43. What is the difference between *authentication* and *authorization*?

Authentication verifies who a user is (e.g., validating credentials or a token). Authorization determines what an authenticated user is allowed to do (e.g., checking roles or claims-based policies before granting access to a resource or action).

Q44. What is *JWT*, and how is it used in .NET Web APIs?

JWT (JSON Web Token) is a compact, self-contained token format used for securely transmitting claims between parties. In .NET Web APIs, after a user authenticates, the server issues a signed JWT

that the client sends in the Authorization header on subsequent requests; the API validates the token's signature and claims to authenticate and authorize the request without needing server-side session state.

Q45. What is the difference between a stored procedure call and LINQ query execution in EF Core?

A LINQ query is translated by EF Core into SQL at run time based on the expression tree, offering flexibility and type safety but sometimes generating less optimal SQL for complex scenarios. Calling a stored procedure directly (via 'FromSqlRaw' or similar) executes precompiled, pre-optimized SQL logic stored in the database, which can be faster for complex operations but loses some of LINQ's composability and type safety.

Q46. What are some common ways to improve performance of a .NET application?

Common approaches include using asynchronous programming for I/O-bound work, enabling response caching and output caching, using 'AsNoTracking()' for read-only EF queries, minimizing unnecessary database round-trips (avoiding N+1 query problems), using efficient data structures, pooling expensive resources like DbContext or HttpClient (via 'IHttpClientFactory'), and profiling to find real bottlenecks rather than guessing.

Q47. What is the difference between unit testing, integration testing, and end-to-end testing in a .NET project?

Unit testing verifies small, isolated pieces of logic (like a single method) usually with mocked dependencies. Integration testing verifies that multiple components work together correctly (e.g., a controller with a real or in-memory database). End-to-end testing verifies the entire application flow from the user's perspective, often through the actual UI or full API surface.

Q48. What is a race condition, and how can it be avoided in .NET?

A race condition occurs when multiple threads access and modify shared data concurrently, and the outcome depends on unpredictable timing, potentially causing bugs. It can be mitigated using synchronization primitives (lock, Monitor, SemaphoreSlim), thread-safe collections (like ConcurrentDictionary), or by avoiding shared mutable state altogether where possible.

Q49. What is the difference between deploying a .NET application as framework-dependent vs self-contained?

A framework-dependent deployment requires the target machine to already have the matching .NET runtime installed, resulting in a smaller deployment package. A self-contained deployment bundles the .NET runtime together with the application, so it can run on a machine without .NET pre-installed, at the cost of a larger deployment size.

Interview Tips

- Be ready to explain concepts with a short real-world example, not just a textbook definition.
- For experienced roles, expect follow-up 'why' questions — know the trade-offs of your answer, not just the answer.
- Practice writing small code snippets on a whiteboard or notepad (without IntelliSense) to build confidence.
- Review the specific .NET version and stack mentioned in the job description — questions often mirror the tech stack used by the team.
- Prepare 2-3 questions of your own about the team's architecture, deployment process, or codebase practices.